



UNIVERSITÀ
DEGLI STUDI
FIRENZE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA
A.A: 2025/2026

SMART LIBRARY

SOFTWARE PER LA GESTIONE DI PRESTITI BIBLIOTECARI

Autore:

ANDREA COFONE

Corso principale:

INGEGNERIA DEL SOFTWARE

N° Matricola:

7118417

Docente corso:

ENRICO VICARIO

REPOSITORY GITHUB:

<https://github.com/andrecofo03/SmartLibrary.git>

Indice

1	Introduzione	3
1.1	Descrizione del progetto	3
1.2	Struttura del progetto	3
2	Progettazione	4
2.1	Class Diagram	4
2.2	Use Case Diagram	5
2.3	Use Case Template	6
2.4	Navigation Diagram	13
2.5	Class Diagram	14
2.5.1	Domain Model	14
2.5.2	Business Logic	15
2.5.3	ORM	15
2.6	ER Diagram	16
2.7	Directory	17
3	Implementazione	19
3.1	Domain Model	19
3.1.1	Utente	19
3.1.2	ElementoBibliotecario	19
3.1.3	Prestito	19
3.1.4	ElementoFactory	19
3.2	Business Logic	20
3.2.1	AuthenticationController	20
3.2.2	StudentLoanController	20
3.2.3	WaitingListManager	20
3.2.4	InputManager e StudentCommandPattern	21
3.3	Object-Relational Mapping	22
3.3.1	DatabaseConnection	22
3.3.2	UserDAO	22
3.3.3	LibraryItemDAO	22
3.3.4	LoanDAO	22
3.3.5	CourseDAO	23
3.4	Database	24
3.4.1	Struttura del Database	24
3.5	Interfaccia	24
3.5.1	Architettura delle view	24
3.5.2	Dettaglio delle interfacce e implementazioni	25
3.6	Design Patterns	25
3.6.1	Singleton	25
3.6.2	Observer	25
3.6.3	Command	25
3.6.4	Factory Method	26
3.6.5	Decorator	26
4	Testing	26
4.1	Test del domain model	26
4.2	Test dei Controller	27
4.3	Test sul DAO	28
4.4	Test sull'Observer	29

1 Introduzione

Progetto realizzato per il superamento del corso di Ingegneria del Software e Laboratorio di Informatica del Corso di Laurea in Ingegneria Informatica. Il progetto consiste in un sistema di classi Java per la simulazione di gestione di un servizio di prestiti di una biblioteca universitaria

1.1 Descrizione del progetto

Il progetto consiste in un sistema avanzato per la gestione di una biblioteca universitaria, che permette agli studenti di ricercare testi accademici specifici presentati per corso di studi e anno, richiedere prestiti di libri cartacei o scaricare ebook. Gli studenti possono inoltre monitorare il proprio storico prestiti, ricevere notifiche in tempo reale sulla disponibilità di libri prenotati e gestire i rinnovi dei prestiti in scadenza, con un limite massimo di estensioni. L'utente amministratore ha il controllo completo sul catalogo, potendo inserire nuovi testi, aggiornare le quantità di copie disponibili, rimuovere risorse dal catalogo

1.2 Struttura del progetto

Il progetto è interamente realizzato in Java. La gestione dei dati è affidata al DBMS PostgreSQL, integrato mediante l'utilizzo del pattern DAO, che permette di separare la logica di business dalla logica di persistenza dei dati. A questo scopo, il progetto è stato strutturato nei package: view (per l'interfaccia CLI e i pattern di input), logic (per i controller e la gestione delle liste d'attesa), model (per le entità e le factory) e orm (per l'accesso al database). La progettazione è stata realizzata attraverso diagrammi Use Case Diagram, Class Diagram ed Entity-Relationship (ER) secondo lo standard UML. Per la fase di testing è stato utilizzato il framework JUnit in combinazione con Mockito

2.2 Use Case Diagram

La progettazione è stata realizzata con uno Use Case Diagram (figura 2), dal quale si notano i ruoli dello studente e dell' amministratore e le operazioni che questi possono fare.

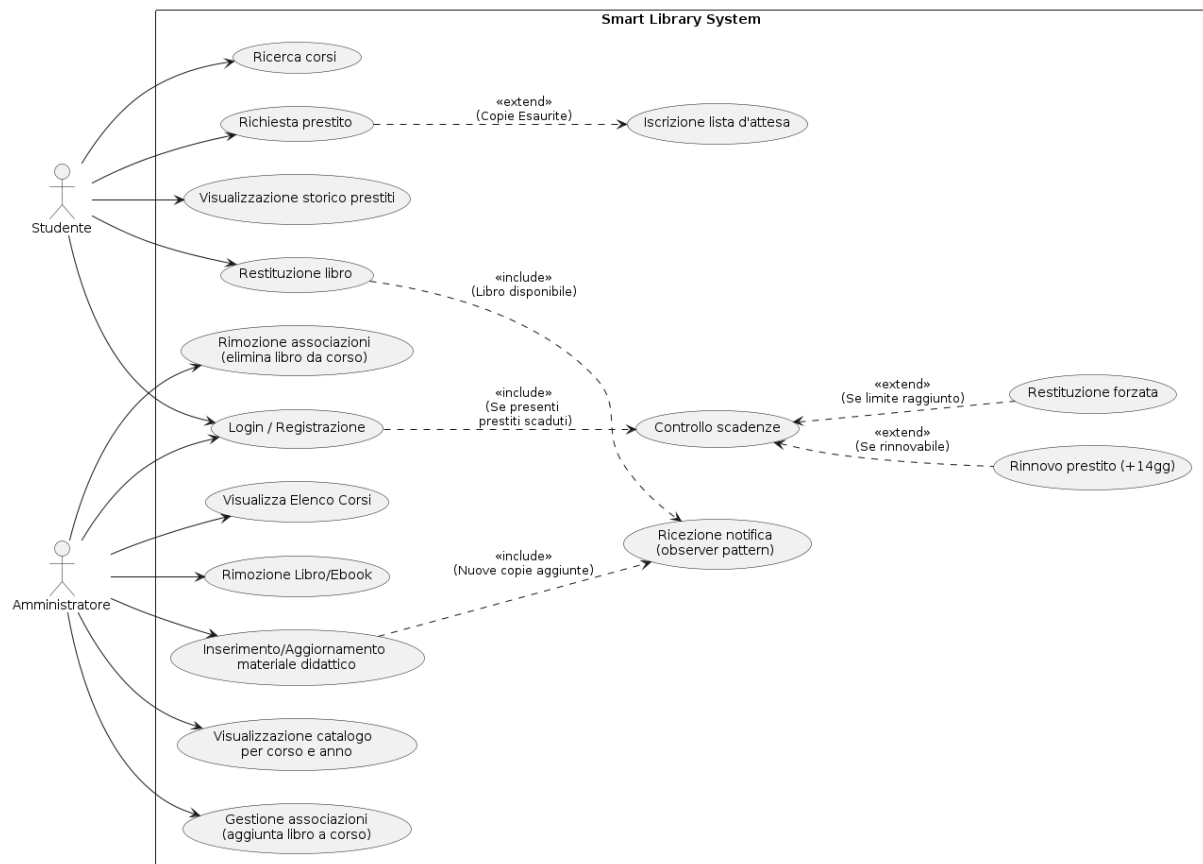


Figura 2: Use Case Diagram

2.3 Use Case Template

Di seguito sono riportati i template di alcuni dei casi d'uso, in ciascuno possiamo trovare: una breve descrizione, il livello del caso d'uso, gli attori coinvolti, le pre-condizioni, le post-condizioni, il flusso base e i flussi alternativi.

Caso d'Uso	Richiesta Prestito Libro Cartaceo
Breve descrizione	Lo studente richiede il prestito di un libro fisico. Se disponibile, viene registrato il prestito e se esaurito, lo studente viene inserito in lista d'attesa.
Livello	User Goal
Attori	Studente
Pre-condizioni	Lo studente è autenticato
Flusso base	1. Lo studente inserisce l'ISBN del libro desiderato 2. Il sistema verifica che venga scelto la tipologia "Libro" corrispondente al libro cartaceo e che le copie disponibili siano maggiori di 0 3. Il sistema decrementa di 1 le copie disponibili 4. Il sistema registra un nuovo prestito con scadenza a 30 giorni 5. Il sistema conferma l'operazione allo studente
Flussi alternativi	• Copie esaurite : Se le copie sono 0, il sistema notifica l'indisponibilità e iscrive automaticamente lo studente alla Lista d'Attesa per quell'ISBN • Libro non trovato: Se l'ISBN non esiste, viene mostrato un messaggio di errore
Post-condizioni	Un nuovo prestito è attivo e il magazzino è aggiornato, oppure lo studente è aggiunto alla lista degli osservatori

Tabella 1: Caso d'uso: Richiesta Prestito

Caso d'Uso	Gestione Scadenze e Rinnovo
Breve descrizione	All'accesso, il sistema rileva prestiti scaduti. Lo studente deve decidere se rinnovare (max 2 volte) o restituire il libro prima di proseguire
Livello	User Goal
Attori	Studente
Pre-condizioni	Lo studente effettua il login ed esiste almeno un prestito scaduto (data scadenza passata e data restituzione nulla).
Flusso base	1. Il sistema blocca l'accesso al menu principale e mostra i prestiti scaduti 2. Il sistema verifica che il contatore rinnovi sia inferiore a 2 3. Lo studente sceglie di rinnovare il prestito 4. Il sistema estende la scadenza di 14 giorni e incrementa il contatore rinnovi 5. L'accesso all'area personale viene sbloccato
Flussi alternativi	• Limite raggiunto: Se il libro è già stato rinnovato 2 volte, il sistema forza la restituzione immediata e chiude il prestito • Rinnovo rifiutato: Se lo studente sceglie di non rinnovare, il sistema registra la restituzione immediata
Post-condizioni	Il prestito ha una nuova data di scadenza posticipata, oppure il prestito viene chiuso definitivamente

Tabella 2: Caso d'uso: Gestione Scadenze

Caso d'Uso	Gestione documenti biblioteca
Breve descrizione	L'amministratore inserisce o aggiorna libri. Se aggiunge copie a un libro esaurito, il sistema notifica automaticamente gli studenti in attesa
Livello	User Goal
Attori	Admin
Pre-condizioni	L'admin è autenticato nel sistema
Flusso base	1. L'admin inserisce un ISBN valido 2. Il sistema rileva che il libro esiste già 3. L'admin inserisce il numero di copie da aggiungere 4. Il sistema aggiorna la quantità totale e disponibile nel database 5. Il sistema invia una notifica agli studenti in lista d'attesa
Flussi alternativi	• Nuovo libro: Se l'ISBN non esiste, il sistema richiede Titolo, Autore, Corso e Anno per creare una nuova voce nel catalogo • Errore tipo: Se si tenta di aggiungere copie fisiche a un record Ebook, il sistema blocca l'operazione
Post-condizioni	Il magazzino è aggiornato e le notifiche di disponibilità sono state inviate agli studenti interessati

Tabella 3: Caso d'uso: Gestione Inventario

Caso d'Uso	Rimozione Risorsa
Breve descrizione	L'amministratore rimuove definitivamente un libro o un ebook dal catalogo, eliminando anche le associazioni ai corsi.
Livello	User Goal
Attori	Admin
Pre-condizioni	L'admin è autenticato
Flusso base	1. L'admin seleziona l'opzione di rimozione e inserisce l'ISBN 2. Il sistema mostra i dettagli del libro e chiede conferma 3. L'admin conferma l'operazione 4. Il sistema elimina il libro e, a cascata, le associazioni ai corsi di studio
Flussi alternativi	• Violazione integrità: Se esistono prestiti attivi non ancora restituiti collegati al libro, il sistema impedisce la cancellazione e mostra un errore • Annullamento: L'admin nega la conferma e l'operazione viene annullata
Post-condizioni	Il libro e i relativi dati storici sono rimossi permanentemente dal database

Tabella 4: Caso d'uso: Rimozione Risorsa

Caso d'Uso	Login e Autenticazione
Breve descrizione	L'utente accede al sistema inserendo le proprie credenziali. Il sistema verifica l'identità, determina il ruolo e controlla eventuali blocchi o scadenze critiche
Livello	User Goal
Attori	Utente Generico (Studente/Admin)
Pre-condizioni	L'applicazione è avviata e connessa al database
Flusso base	1. L'utente inserisce matricola e password 2. Il sistema verifica la corrispondenza delle credenziali nel database 3. Il sistema identifica il ruolo 4. Se Studente: Il sistema esegue il controllo automatico delle scadenze 5. Il sistema concede l'accesso al menu principale specifico per il ruolo
Flussi alternativi	• Credenziali errate: Il sistema mostra un messaggio di errore e richiede nuovamente l'input
Post-condizioni	L'utente è autenticato e la sessione corrente contiene i suoi dati

Tabella 5: Caso d'uso: Login

Caso d'Uso	Consultazione catalogo
Breve descrizione	Lo studente visualizza i libri adottati nel proprio corso di laurea
Livello	User Goal
Attori	Studente, Sistema
Pre-condizioni	Lo studente è autenticato
Flusso base	1. Lo studente seleziona l'opzione "Visualizza Libri" 2. Il sistema mostra l'elenco dei Corsi di Laurea disponibili 3. Lo studente seleziona il proprio corso 4. Il sistema chiede l'anno di interesse 5. Il sistema esegue una query 6. Il sistema mostra l'elenco dei libri con dettagli (titolo, autore, ISBN, disponibilità)
Flussi alternativi	• Nessun risultato: Se non ci sono libri associati a quel corso/anno, il sistema mostra un messaggio informativo • Annullamento: Lo studente torna al menu principale senza effettuare ricerche
Post-condizioni	I risultati della ricerca sono visualizzati a schermo

Tabella 6: Caso d'uso: Ricerca Guidata

Caso d'Uso	Restituzione libro
Breve descrizione	Lo studente restituisce un libro cartaceo preso in prestito. Il sistema aggiorna l'inventario e notifica automaticamente gli utenti in lista d'attesa
Livello	User Goal
Attori	Studente
Pre-condizioni	Lo studente ha almeno un prestito attivo per il libro che vuole restituire
Flusso base	1. Lo studente seleziona l'opzione "Restituzione" e inserisce l'ISBN 2. Il sistema verifica che esista un prestito attivo per quell'utente e quel libro 3. Il sistema registra la data di restituzione (chiude il prestito) 4. Il sistema incrementa di 1 le copie disponibili nel magazzino 5. Il sistema controlla la Lista d'Attesa per quell'ISBN 6. Se ci sono osservatori, il sistema invia una notifica di disponibilità
Flussi alternativi	• Prestito non trovato: L'ISBN inserito non corrisponde a nessun prestito attivo dell'utente • Restituzione ebook: Se l'ISBN è un Ebook, il sistema revoca l'accesso senza incrementare copie fisiche
Post-condizioni	Il prestito è chiuso, l'inventario è aggiornato

Tabella 7: Caso d'uso: Restituzione libro

2.4 Navigation Diagram

Il diagramma di navigazione (figura 3) descrive la struttura dell'applicazione, suddividendo le funzionalità tra studenti e amministratori. Dalla schermata iniziale di login, il sistema indirizza automaticamente l'utente verso l'area di competenza in base al ruolo rilevato (Studente o Admin). L'Area studente permette di ricercare libri filtrati per corso di laurea e anno accademico, richiedere prestiti, visualizzare lo storico personale e gestire le scadenze critiche. In caso di prestiti scaduti, il flusso viene interrotto forzatamente per richiedere un rinnovo o una restituzione immediata prima di accedere alle altre funzioni. L'Area amministratore offre strumenti per la gestione completa del catalogo (inserimento, modifica quantità, rimozione risorse), con notifiche automatiche agli studenti in attesa. Ogni sezione include sempre la possibilità di tornare al menu principale o effettuare il logout, garantendo un flusso intuitivo. I numeri scritti sopra le frecce corrispondono ai comandi da digitare tramite la CLI.

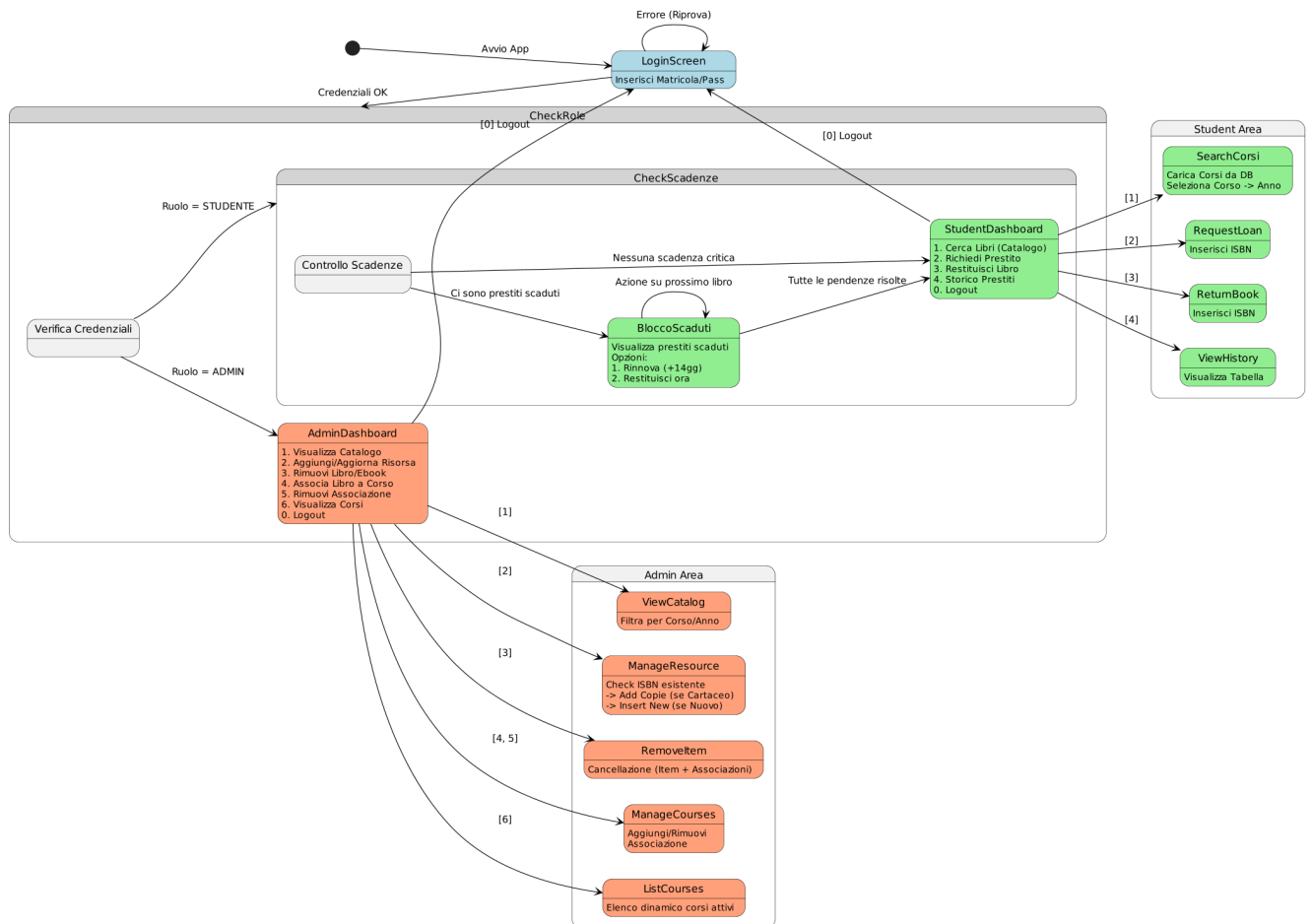


Figura 3: Navigation Diagram

2.5 Class Diagram

Il progetto è strutturato in package distinti, con lo scopo di separare le responsabilità e seguire il principio "Separation of Concerns". L'obiettivo è di avere un codice che sia facilmente estendibile con nuove funzionalità e che possa essere intuitivo all'uso

2.5.1 Domain Model

Il livello Domain Model ha la responsabilità di rappresentare le entità principali del sistema (figura 4). Definisce le classi core come *Studente*, *Admin* e la gerarchia di *ElementoBibliotecario*. Include inoltre la classe *Prestito* per la gestione dello storico e l'interfaccia *Observer* utilizzata per il pattern omonimo, permettendo agli oggetti di reagire ai cambiamenti di stato (es. disponibilità di un libro)

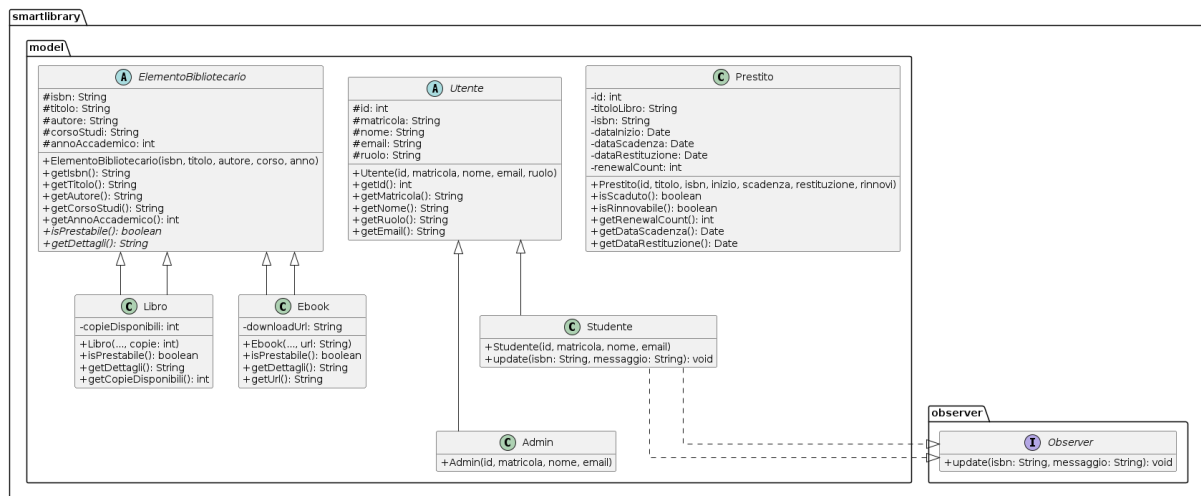


Figura 4: Domain Model

2.5.2 Business Logic

La Business Logic (figura 5) gestisce il comportamento principale del sistema, coordinando le operazioni tra l'interfaccia utente e il livello dei dati. La sua responsabilità è guidare i flussi di lavoro come la ricerca guidata, l'assegnazione dei prestiti, la gestione delle code di attesa e la validazione delle regole di business. I Controller sono organizzati per attore e funzionalità (ad esempio StudentLoanController gestisce la logica dei prestiti per lo studente, mentre la logica di input e comandi è delegata a classi specifiche nel package View, che interagiscono strettamente con la logica)

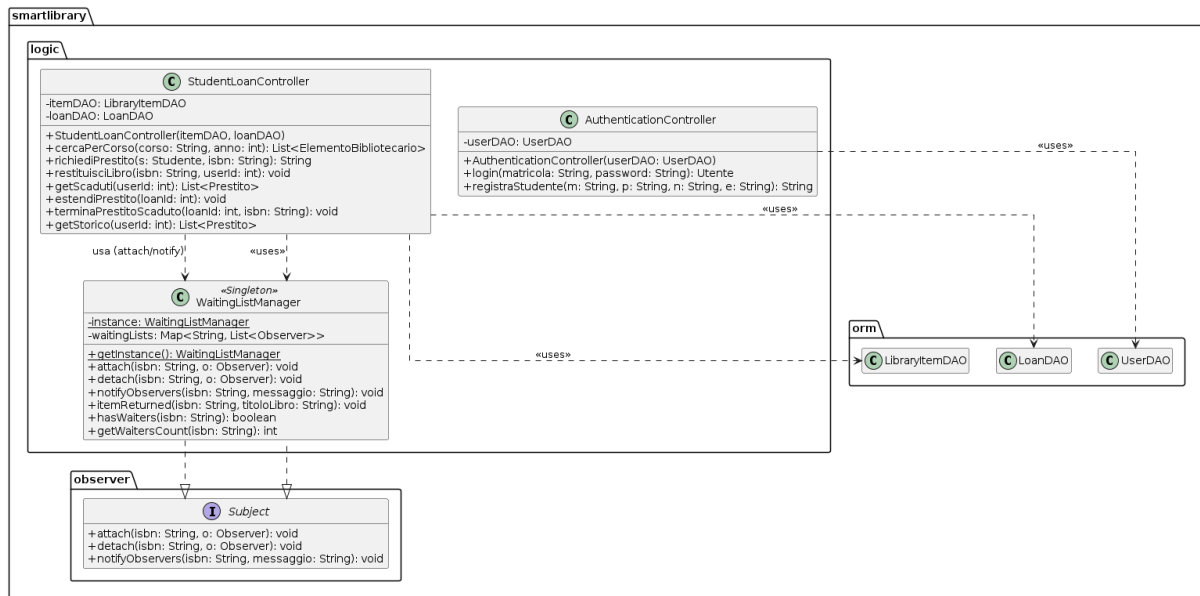


Figura 5: Business Logic

2.5.3 ORM

Il livello ORM (figura 6) ha la responsabilità di gestire l'interazione tra l'applicazione e il database. Fornisce metodi per accedere, salvare, modificare ed eliminare dati riguardanti utenti, ordini, articoli, metodi di pagamento e amministratori. Utilizza un singleton chiamato ConnectionManager per garantire una gestione centralizzata della connessione al database, possiamo dire quindi che questo livello traduce gli oggetti del dominio in dati persistenti nel database

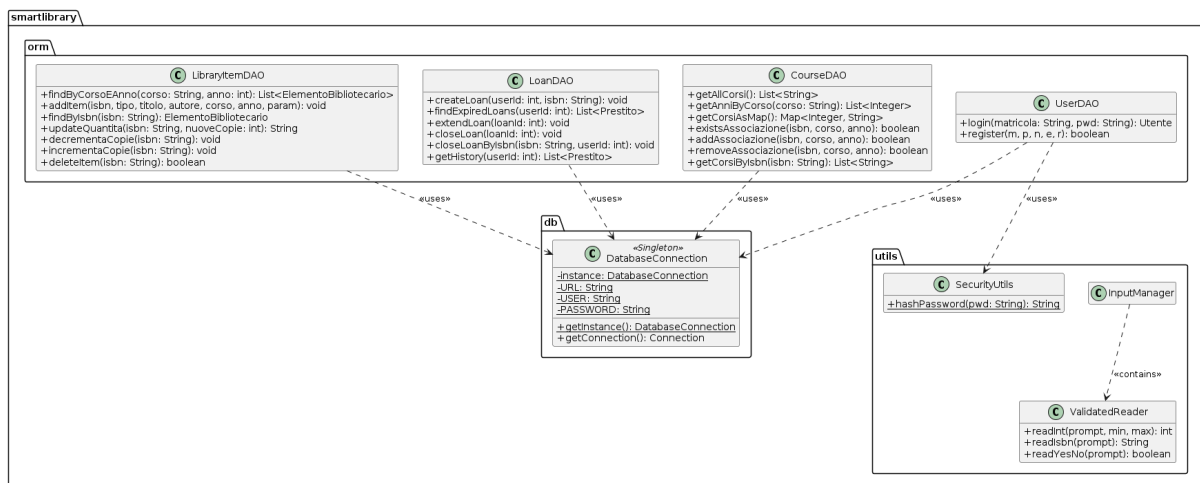


Figura 6: ORM

2.6 ER Diagram

La progettazione del database è stata realizzata attraverso uno schema concettuale ER (Entity-Relationship, figura 7). In questo diagramma sono raffigurate tutte le entità principali del sistema e le relative relazioni. Lo schema rappresenta la struttura logica del database prima della successiva fase di implementazione "fisica" in PostgreSQL, per la quale abbiamo invece usato uno schema relazionale

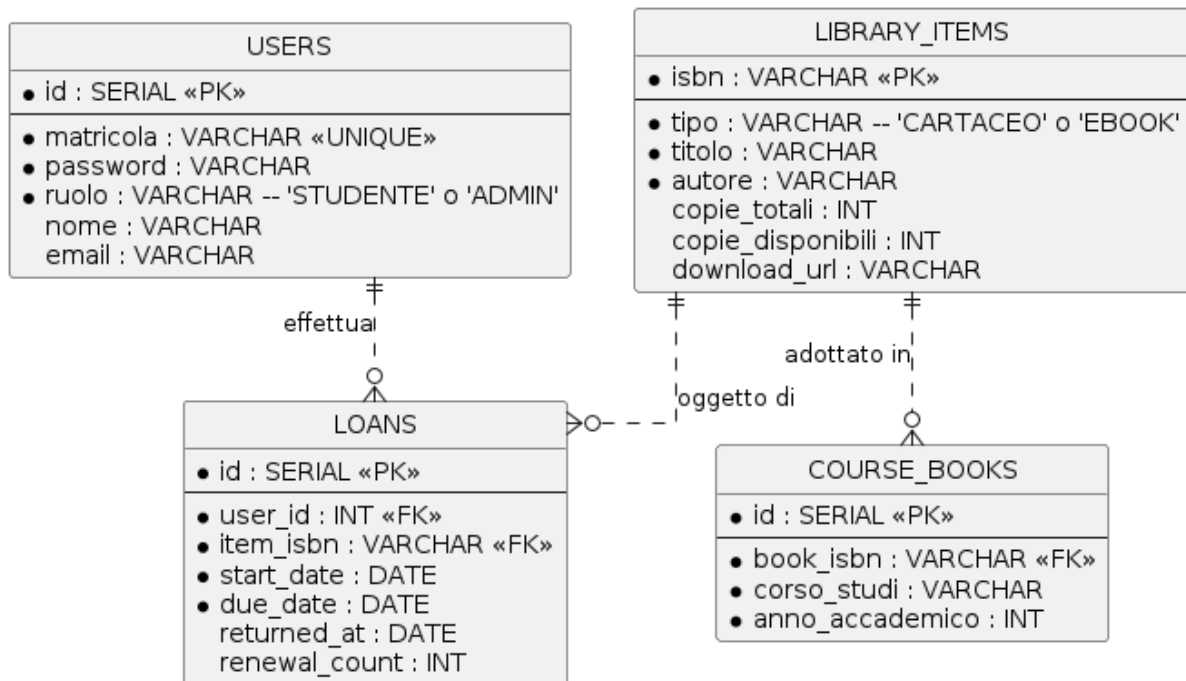


Figura 7: ER Diagram

2.7 Directory

Il progetto è organizzato seguendo la struttura standard di Maven, con una suddivisione dei package che ha come scopo quello di separare le responsabilità dei vari elementi del progetto. Le principali directory sono:

- **src/main/java/com/smartlibrary:** Directory radice contenente tutto il codice sorgente dell'applicazione
 - **db:** Contiene la classe Singleton per la gestione della connessione al database PostgreSQL
 - **factory:** Contiene le classi Factory per la creazione dinamica degli oggetti
 - **logic:** Contiene i Controller che gestiscono la logica di business e il coordinamento dei flussi
 - **model:** Contiene le classi che rappresentano le entità del dominio
 - **observer:** Contiene le interfacce Observer e Subject per l'implementazione del pattern observer
 - **orm:** Contiene le classi DAO responsabili dell'interazione diretta con il database
 - **utils:** Contiene classi che aggiungono dettaglio al progetto, come la validazione dell'input e la gestione delle password
 - **view:** Contiene le classi relative all'interfaccia utente a riga di comando (CLI) e l'implementazione del pattern Command
- **src/main/sql:** Contiene il file di configurazione del database di PostgreSQL che permette la creazione delle tabelle necessarie e con l'inserimento di alcuni esempi di dati da inserire per controllare il corretto funzionamento del programma
- **src/test/java/com/smartlibrary:** Contiene le classi dedicate allo Unit Testing per verificare il corretto funzionamento delle singole classi in casi d'uso limite e per accertarne il corretto funzionamento

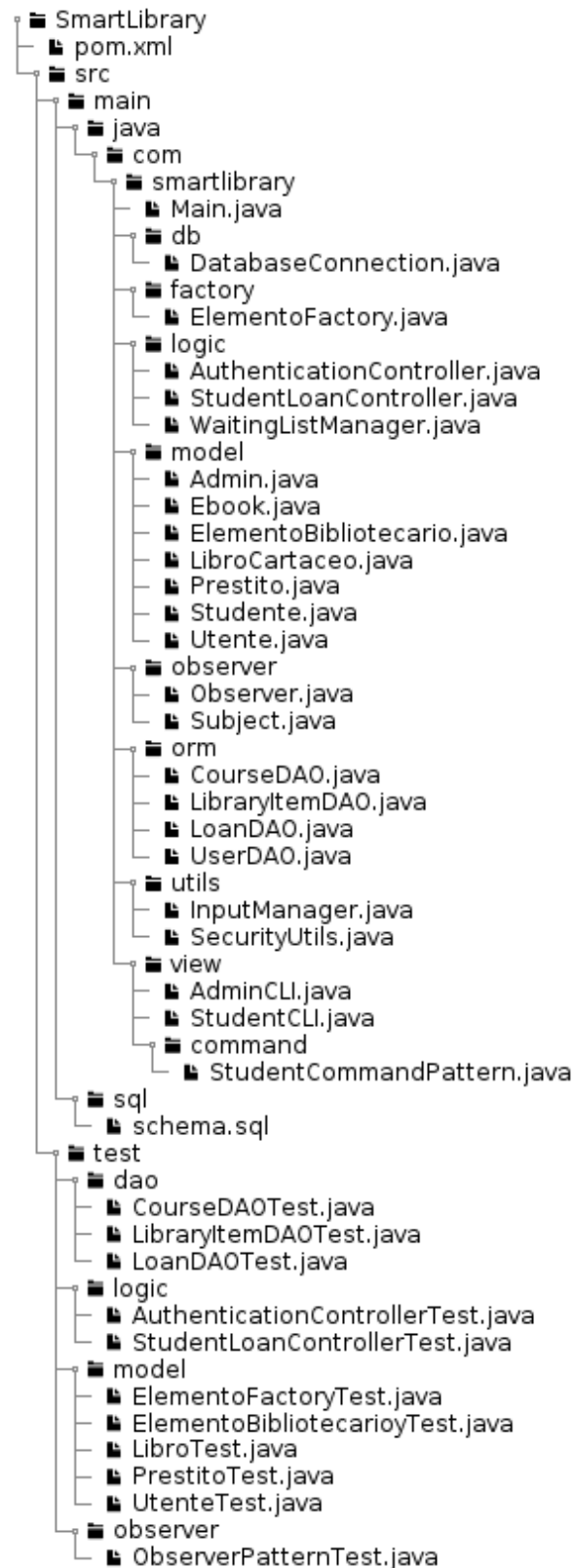


Figura 8: Directory progetto

3 Implementazione

3.1 Domain Model

3.1.1 Utente

La classe astratta `Utente` rappresenta l'entità base del sistema, definita dagli attributi: `id` (univoco, generato dal database), `matricola` (identificativo per il login), `nome`, `email` e `ruolo`. Da questa classe derivano due specializzazioni:

- **Studiante**: Implementa l'interfaccia `Observer` per ricevere notifiche sulla disponibilità dei libri tramite il metodo `update()`. È l'attore principale che richiede prestiti, consulta il catalogo e gestisce i rinnovi
- **Admin**: Rappresenta l'amministratore del sistema, dotato di privilegi per la gestione del catalogo, delle associazioni libro-corso e della visualizzazione dei corsi di studio

3.1.2 ElementoBibliotecario

La classe astratta `ElementoBibliotecario` rappresenta la risorsa generica della biblioteca, definita dagli attributi: `isbn`, `titolo`, `autore`, `corsoStudi` e `annoAccademico`. I metodi astratti `isPrestabile()` e `getDettagli()` sono implementati dalle due sottoclassi:

- **Libro**: Rappresenta un libro cartaceo. Aggiunge l'attributo `copieDisponibili` e implementa `isPrestabile()` verificando che le copie disponibili siano maggiori di zero. Il metodo `getDettagli()` restituisce una stringa formattata con il prefisso `[CARTACEO]` e il numero di copie disponibili
- **Ebook**: Rappresenta una risorsa digitale. Aggiunge l'attributo `downloadUrl` e implementa `isPrestabile()` ritornando sempre `true`, in quanto risorsa illimitata. Il metodo `getDettagli()` restituisce una stringa con il prefisso `[EBOOK]`

3.1.3 Prestito

La classe `Prestito` rappresenta l'associazione temporale tra uno `Studiante` e un `ElementoBibliotecario`. È definita dagli attributi: `id`, `titoloLibro`, `isbn`, `dataInizio`, `dataScadenza`, `dataRestituzione` e `renewalCount` (contatore dei rinnovi effettuati). La classe implementa due metodi di verifica che governano le regole di business sui prestiti:

- `isScaduto()`: Verifica se la data corrente supera la data di scadenza e il campo `dataRestituzione` è `null` (prestito non ancora restituito). Un prestito restituito non risulta mai scaduto, anche se la data limite è passata
- `isRinnovabile()`: Verifica se il `renewalCount` è inferiore al limite massimo consentito, impostato a 2. Questo permette allo studente di estendere il prestito al massimo due volte prima della restituzione obbligatoria

3.1.4 ElementoFactory

La classe `ElementoFactory` implementa il design pattern **Factory Method**. Offre un metodo statico `creaElemento()` che, ricevendo il tipo e i parametri grezzi provenienti dal database o dall'input utente, istanzia e restituisce l'oggetto concreto corretto. Il confronto sul tipo avviene tramite `equalsIgnoreCase`, rendendo il metodo case-insensitive. In caso di tipo non riconosciuto, la factory lancia un `IllegalArgumentException`, impedendo la creazione di oggetti incoerenti

3.2 Business Logic

3.2.1 AuthenticationController

Questa classe gestisce il flusso di autenticazione e registrazione degli utenti. Riceve il **UserDAO** tramite dependency injection nel costruttore, permettendo l'iniezione di mock durante i test. Il metodo `login()` verifica che matricola e password non siano `null`, poi delega al DAO la verifica delle credenziali, che avviene confrontando l'hash della password. Il metodo `registraStudente()` applica la validazione prima dell'accesso al database: la matricola deve corrispondere esattamente a 7 cifre e la password deve avere almeno 5 caratteri. Solo dopo la validazione viene invocato il DAO per l'inserimento, con il ruolo aggiunto dal controller

3.2.2 StudentLoanController

Questa classe coordina le operazioni dello studente, gestendo il ciclo completo del prestito. Riceve **LibraryItemDAO** e **LoanDAO** tramite costruttore, i metodi principali sono:

- `cercaPerCorso()`: Filtra il catalogo in base al corso di studi e all'anno accademico, delegando la query al **LibraryItemDAO**
- `richiediPrestito()`: Verifica la disponibilità della risorsa e gestisce tre scenari distinti. Per un libro cartaceo disponibile, decrementa le copie e crea il record prestito. Per un ebook, crea il record prestito per gestire la scadenza dell'accesso senza modificare l'inventario
- `restituisceLibro()`: Chiude il prestito tramite `closeLoanByIsbn` e, se la risorsa è un libro cartaceo, incrementa le copie disponibili
- `estendiPrestito()`: Delega al **LoanDAO** l'estensione della scadenza di 14 giorni e l'incremento del contatore rinnovi
- `getScaduti()`: Recupera i prestiti scaduti non restituiti per lo studente, utilizzati dalla **StudentCLI** per bloccare l'accesso al menu principale finché i prestiti critici non vengono gestiti
- `terminaPrestitoScaduto()`: Chiude forzatamente un prestito scaduto e, se cartaceo, ripristina l'inventario e notifica la lista d'attesa, simulando una restituzione automatica da parte del sistema

3.2.3 WaitingListManager

Questa classe implementa contemporaneamente i pattern **Singleton** e **Observer**, fungendo da **Subject**. Gestisce una mappa che associa ogni ISBN alla lista degli studenti in attesa. L'interfaccia **Subject** è stata specializzata per il dominio con i seguenti metodi:

- `attach()`: Registra uno studente nella lista d'attesa per un libro specifico
- `detach()`: Rimuove uno studente dalla lista d'attesa di un ISBN specifico. Se la lista resta vuota, rimuove anche la chiave dalla mappa
- `notifyObservers()`: Notifica tutti gli observer registrati per un ISBN
- `itemReturned()`: Metodo che compone il messaggio di disponibilità, invoca `notifyObservers` e svuota la coda per quell'ISBN
- `hasWaiters()` e `getWaitersCount()`: Metodi per verificare lo stato delle code, utilizzabili per mostrare informazioni sulle liste d'attesa

3.2.4 InputManager e StudentCommandPattern

Queste classi, parte del layer View/Utility, contengono logica di validazione e organizzazione dei flussi dell'interfaccia utente:

- **InputManager** implementa il pattern **Decorator**. L'implementazione **ConsoleReader** legge da tastiera tramite **Scanner**. La classe **ValidatedReader** wrappa un **InputReader** e costruisce sopra al metodo base i metodi di validazione: **readInt()** per interi con range min-max, **readIsbn()** per ISBN a 13 cifre, **readText()** per stringhe non vuote e non esclusivamente numeriche, e **readYesNo()** per conferme s/n. In caso di input errato, ogni metodo richiede automaticamente un nuovo inserimento
- **StudentCommandPattern** implementa il pattern **Command**. L'interfaccia **Command** definisce i metodi **execute()** e **getDescription()**. Le classi **SearchCommand**, **LoanCommand**, **ReturnCommand** e **HistoryCommand** incapsulano le azioni dello studente in oggetti, permettendo alla **StudentCLI** di costruire il menu per poter essere esteso con nuove funzionalità senza modificare la struttura della classe **CLI**

3.3 Object-Relational Mapping

3.3.1 DatabaseConnection

La classe `DatabaseConnection` implementa il design pattern **Singleton** per garantire una gestione centralizzata della connessione al database PostgreSQL. Il costruttore stabilisce la connessione tramite `DriverManager` con le credenziali configurate (URL, user, password). Il metodo statico `getInstance()` verifica lo stato della connessione tramite `isClosed()`, ricreandola se necessario. L'oggetto `Connection` viene fornito ai vari DAO tramite il metodo `getConnection()`

3.3.2 UserDAO

La classe `UserDAO` gestisce la persistenza degli utenti sulla tabella `users`. Il metodo `login()` esegue una query confrontando matricola e hash della password. Il `ResultSet` viene mappato nell'oggetto leggendo la colonna `ruolo`: se il valore è "ADMIN" viene istanziato un oggetto `Admin`, altrimenti uno `Studente`. Il metodo `register()` esegue una `INSERT` con l'hash della password calcolato tramite `hashPassword()`, e restituisce `false` in caso di matricola duplicata

3.3.3 LibraryItemDAO

Gestisce le operazioni CRUD sulla tabella `library_items` e interagisce con la tabella `course_books` per le associazioni. I metodi principali sono:

- `findByCorsoEAnno()`: Esegue una `JOIN` tra `library_items` e `course_books` per filtrare i libri in base al corso di studi e all'anno accademico,
- `findByIsbn()`: Cerca un singolo libro per ISBN nella tabella `library_items`
- `addItem()`: Gestisce l'inserimento del libro in `library_items` e la creazione dell'associazione in `course_books`
- `updateQuantita()`: Aggiorna `copie_totali` e `copie_disponibili`, restituendo il titolo del libro per consentire l'invio delle notifiche Observer agli studenti in lista d'attesa
- `deleteItem()`: Rimuove una risorsa dalla tabella `library_items`. Le associazioni nella tabella ponte vengono eliminate automaticamente dal database. Se esistono prestiti attivi collegati all'ISBN, il metodo restituisce `false`

3.3.4 LoanDAO

La classe `LoanDAO` gestisce la tabella `loans`, che registra il ciclo di vita completo dei prestiti. Ogni metodo esegue operazioni su SQL:

- `createLoan()`: Inserisce un nuovo record con data di inizio prestito impostata a `CURRENT_DATE` e data di scadenza calcolata come `CURRENT_DATE + 30`, definendo una durata standard di 30 giorni
- `findExpiredLoans()`: Esegue una `JOIN` con `library_items` per recuperare i prestiti attivi la cui scadenza è precedente alla data corrente, identificando i prestiti scaduti non restituiti
- `extendLoan()`: Aggiorna atomicamente la data di scadenza aggiungendo 14 giorni e incrementa il `renewal_count` di 1, in una singola istruzione `UPDATE`
- `closeLoan()`: Imposta la data di ritorno di un prestito a `CURRENT_DATE` per un dato id di prestito, terminandolo

- `closeLoanByIsbn()`: Cerca il prestito attivo per ISBN e ID utente, dove lo studente inserisce l'ISBN
- `getHistory()`: Recupera lo storico dei prestiti di un utente ordinato per data di inizio decrescente

3.3.5 CourseDAO

La classe `CourseDAO` gestisce l'accesso alla tabella `course_books`, che risolve la relazione molti-a-molti tra libri e corsi di studio. I metodi di lettura sono:

- `getAllCorsi()`: Recupera tutti i corsi di studio distinti tramite `SELECT DISTINCT`, ordinati alfabeticamente
- `getAnniByCorso()`: Recupera gli anni accademici disponibili per un dato corso, ordinati in modo crescente
- `getCorsiAsMap()`: Restituisce una mappa con indici che vengono rappresentati nei menu numerati delle CLI
- `getCorsiByIsbn()`: Recupera tutti i corsi associati a un ISBN, per mostrare all'admin in quali corsi è adottato un libro

I metodi di scrittura sono:

- `existsAssociazione()`: Verifica l'esistenza di un'associazione prima dell'inserimento, prevenendo duplicati
- `addAssociazione()`: Inserisce una nuova associazione libro-corso dopo aver verificato che non esista già, restituendo `false` in caso di duplicato
- `removeAssociazione()`: Rimuove un'associazione specifica

3.4 Database

Il database PostgreSQL è stato progettato per garantire l'integrità referenziale e gestire le relazioni tra utenti, libri e prestiti

3.4.1 Struttura del Database

Come illustrato nello schema Entity-Relationship (ER), lo schema prevede quattro tabelle principali:

- **users:** Memorizza le informazioni di tutti gli attori del sistema. Utilizza una colonna discriminante **ruolo** per mappare la gerarchia delle classi Java in un'unica tabella. Le password vengono salvate esclusivamente sotto forma di hash per simulare la gestione di esse con una maggior sicurezza
- **library_items:** Contiene il catalogo completo delle risorse, inoltre, gestisce sia i Libri che gli Ebook tramite una colonna **tipo**
 - Per i libri, vengono popolati i campi **copie_totali** e **copie_disponibili**
 - Per gli ebook, questi campi sono nulli o zero, mentre viene riempito il campo **download_url**
- **course_books:** Risolve la relazione molti-a-molti tra i libri e i corsi di studio, inoltre siccome un testo può essere adottato in più corsi di laurea e in anni accademici diversi, questa tabella collega l'ISBN del libro al nome del corso e all'anno
- **loans:** Registra lo storico e lo stato attuale dei prestiti. Oltre alle date di inizio e scadenza, include:
 - **returned_at:** Se null, il prestito è attivo, altrimenti, il libro è stato restituito
 - **renewal_count:** Un contatore per applicare la regola che limita i rinnovi a un massimo di due volte

I file SQL relativi alla creazione e popolazione del database sono:

- **schema.sql:** Contiene le istruzioni per cancellare le tabelle esistenti e ricrearle con i vincoli corretti, inoltre contiene degli esempi per l'inserimento di dati utili per testare le funzionalità dell'applicazione

3.5 Interfaccia

Per l'applicazione è stata realizzata una Command Line Interface (CLI). Questa scelta ha permesso di dare attenzione allo sviluppo della logica applicativa e delle interazioni del dominio. Si ha sempre la possibilità di una possibile evolutiva andando a implementare l'interfaccia grafica vera e propria (GUI), permettendo una miglior gestione degli elementi dell'applicazione

3.5.1 Architettura delle view

Le interfacce utente sono state progettate per essere separate dalla logica di business. Questa organizzazione implementa una separazione tra la CLI e il Controller, migliorando la gestione di essa. L'output verso l'utente viene effettuato tramite i metodi di stampa a schermo e per la formattazione tabellare dei dati, mentre l'input viene validato attraverso la classe decorator appartenente a **InputManager** che incapsula la lettura da console e garantisce la correttezza dei dati inseriti

3.5.2 Dettaglio delle interfacce e implementazioni

Main

La classe **Main** funge da punto di ingresso dell'applicazione. Si occupa della presentazione del menu iniziale di autenticazione e dirige il flusso dell'utente verso la CLI appropriata in base al ruolo rilevato

StudentCLI La **StudentCLI** gestisce l'interazione con gli studenti. All'avvio, esegue un controllo bloccante sulle scadenze critiche tramite il metodo `checkScadenzeCritiche()`, impedendo l'accesso al menu principale se vi sono prestiti scaduti non gestiti.

AdminCLI La **AdminCLI** riguarda le operazioni di gestione. Consente all'amministratore di visualizzare il catalogo filtrato per corso di laurea, inserire nuovi libri/ebook e rimuovere risorse. Si ha un sistema di feedback immediato, come la conferma dell'invio di notifiche agli utenti in lista d'attesa quando viene incrementata la disponibilità di un libro

InputManager Questa classe agisce fornisce metodi di utilità e viene utilizzata nella classe **Scanner**. Tra alcuni metodi presenti vediamo `readIsbn()` (che valida il formato a 13 cifre), `readInt()` (con limiti min/max) e `readYesNo()`. In caso di input errato, gestisce l'eccezione internamente e richiede nuovamente l'input all'utente

3.6 Design Patterns

3.6.1 Singleton

Il Singleton è un pattern creazionale che ha lo scopo di garantire che una classe venga istanziata una sola volta nel ciclo di vita dell'applicativo e che l'istanza sia accessibile globalmente. In questo progetto è stato utilizzato per le classi **DatabaseConnection** e **WaitingListManager**

DatabaseConnection È la classe che si occupa di gestire la connessione al database PostgreSQL. L'obiettivo è assicurarsi che ogni accesso ai dati (dai vari DAO) utilizzi sempre la stessa istanza di **Connection**, ottimizzando le risorse e centralizzando la configurazione. Il metodo `getInstance()` restituisce l'unica istanza attiva

WaitingListManager Questa classe funge da registro centrale per le code d'attesa dei libri. Essendo che deve essere accessibile sia dai controller degli studenti (per iscriversi) sia da quelli admin (per notificare), il Singleton garantisce che lo stato delle code condiviso è disponibile sempre durante l'uso dell'applicazione

3.6.2 Observer

Nel progetto è stato implementato per gestire le notifiche di disponibilità dei libri:

- **Subject:** La classe **WaitingListManager** mantiene una mappa di liste d'attesa. Quando un libro torna disponibile (tramite restituzione o nuova disponibilità), il metodo `itemReturned()` notifica tutti gli osservatori interessati a quello specifico ISBN
- **Observer:** La classe **Studente** implementa l'interfaccia **Observer**. Il metodo `update()` riceve il messaggio di notifica, simulando l'invio di un avviso tramite stampa su schermo

3.6.3 Command

Il pattern Command è stato utilizzato nella **StudentCLI** per incapsulare le richieste dell'utente come oggetti. Le classi come **SearchCommand**, **LoanCommand** e **ReturnCommand** implementano un'interfaccia comune. Questo ha permesso di disaccoppiare la logica di visualizzazione del

menu dall'esecuzione effettiva delle operazioni, facilitando l'aggiunta di nuove funzionalità senza modificare la struttura della classe CLI

3.6.4 Factory Method

Il pattern Factory Method è stato impiegato nella classe `ElementoFactory` per delegare la creazione degli oggetti `ElementoBibliotecario`. A seconda dei dati recuperati dal database, la factory istanzia e restituisce un oggetto `Libro` o `Ebook`, andando a facilitare l'istanziamento e i dettagli specifici delle sottoclassi alle classi DAO

3.6.5 Decorator

Il pattern Decorator è stato implementato nella classe `ValidatedReader` in modo da permettere di aggiungere responsabilità a un oggetto in modo dinamico, wrappando un componente base con funzionalità aggiuntive, senza modificare il componente originale. Il decorator riceve un `InputReader` nel costruttore e lo salva come campo interno `wrapped`. Il metodo `readLine` agisce direttamente al componente wrappato, mantenendo la compatibilità con l'interfaccia base, qui abbiamo l'implementazione del design pattern per il quale si hanno metodi aggiuntivi *sopra* a `readLine`: ad esempio, `readInt` chiama ripetutamente `readLine` finché l'utente non inserisce un intero nel range richiesto, `readIsbn` chiama `readLine` finché il formato non è valido, oppure `readYesNo` chiama `readLine` finché la risposta non è "s" o "n"

4 Testing

Sono stati realizzati Unit Test per verificare la correttezza della logica di business e delle funzionalità critiche, isolando i componenti da dipendenze esterne complesse dove necessario. Sono state utilizzate le librerie **JUnit 5** per la struttura dei test. Gli Unit Test verificano che una porzione di codice funzioni correttamente in isolamento

4.1 Test del domain model

ElementoFactoryTest Testa il pattern **Factory Method** implementato nella classe `ElementoFactory`, verificando che la creazione degli oggetti di dominio avvenga correttamente in base al tipo specificato.

- **testFactoryCreationEbook**: Invoca `ElementoFactory.creaElemento()` con tipo "EBOOK" e verifica che l'oggetto restituito sia un'istanza di `Ebook`. Controlla inoltre che gli attributi di dominio (`corsoStudi` e `annoAccademico`) siano mappati correttamente e che `isPrestabile()` restituisca `true`, confermando l'invariante della disponibilità illimitata degli ebook
- **testFactoryCreationLibro**: Invoca la factory con tipo "CARTACEO" e verifica che l'oggetto restituito sia un'istanza di `Libro`. Controlla che l'anno accademico venga preservato durante il processo di creazione, validando il corretto passaggio dei parametri dal metodo `factory` al costruttore della classe concreta
- **testFactoryTipoSconosciuto_LanciaEccezione**: Invoca la factory con un tipo non riconosciuto e verifica che venga rilevato come errore. Questo test garantisce che vengano rifiutati tipi non previsti, impedendo la creazione di oggetti non riconosciuti nell'applicazione

PrestitoTest La classe `PrestitoTest` verifica la macchina a stati interni dei prestiti

- **testIsScaduto:** Verifica la logica temporale coprendo tre scenari: scadenza passata senza restituzione (true), data futura (false) e scadenza passata ma con libro restituito (false).
- **testIsRinnovabile:** Verifica il vincolo sul numero massimo di rinnovi, assicurando che un prestito con `renewalCount` ≥ 2 non possa essere esteso ulteriormente

LibroTest Si concentra sulla gestione dell'inventario fisico.

- **testIsPrestabile:** Assicura che la disponibilità del libro dipenda strettamente dal contatore `copieDisponibili`. Se il valore è zero, il metodo deve restituire **false**, impedendo il prestito nel controller
- **testGetDettagli:** Verifica che la formattazione della stringa descrittiva includa il tipo di risorsa (cartaceo/ebook), titolo e il numero di copie residue

UtenteTest Verifica la gerarchia di ereditarietà tra **Utente**, **Studente** e **Admin**.

- **testRuoliCorretti:** Verifica che il costruttore di **Studente** imposti automaticamente il ruolo a "STUDENTE" e quello di **Admin** a "ADMIN", garantendo la sicurezza nelle logiche di autorizzazione.
- **testStudenteImplementaObserver:** Verifica tramite `instanceof` che la classe **Studente** implementi correttamente l'interfaccia **Observer**, caratteristica che è necessaria per il sistema di notifiche della lista d'attesa

ElementoBibliotecarioTest

- **testLibroDisponibilita:** Verifica che `isPrestabile()` ritorni true con copie > 0 e false con copie = 0
- **testEbookDisponibile :** Verifica che un ebook sia sempre prestabile, indipendentemente da qualsiasi parametro

4.2 Test dei Controller

StudentLoanControllerTest Testa il controller che coordina la logica dei prestiti

- **testRichiestaPrestitoCartaceo:** Simula un libro cartaceo con delle copie disponibili tramite e dopo la richiesta di prestito, verifica tre effetti: il messaggio di ritorno contenga la conferma del prestito, decrementa le copie del libro e crea un prestito con l'ID dello studente e l'ISBN. Tutti e tre devono avvenire affinché il prestito sia considerato corretto
- **testRichiestaPrestitoCartaceoEsaurito:** Simula un libro senza copie, si verifica che il messaggio indichi che abbiamo copie esaurite e che non venga eseguito il prestito e non si decrementi la quantità del libro, non essendo disponibile
- **testRichiestaPrestitoEbook:** Si verifica che il messaggio contenga l'url scaricabile, venga invocata la creazione del prestito per registrare la scadenza dell'accesso, senza chiamare il decremento di copie

- **testRichiestaPrestitoIsbnNonTrovato:** Verifica che il controller comunica il non aver trovato il libro e la richiesta del prestito e il decremento delle copie non vengano mai invocati
- **testRestituisciLibroCartaceo:** Dopo l'invocazione della restituzione del libro, verifica le due operazioni necessarie: venga chiamato il metodo di chiusura prestito per ISBN con ISBN e ID utente, e l'incremento delle copie viene invocato per ripristinare le quantità disponibili
- **testTerminaPrestitoScadutoCartaceo:** Verifica la chiusura forzata di un prestito scaduto simulando una restituzione automatica da parte del sistema per scadenza dei termini. Viene chiamato il metodo di terminazione del prestito scaduto con un ID prestito e un ISBN, poi verifica che il metodo di chiusura del prestito venga invocato con l'ID del prestito e che l'incremento delle copie venga invocato sull'ISBN

AuthenticationControllerTest Testa la logica di autenticazione e registrazione

- **testLoginSuccess:** Il test verifica che il controller propaghi correttamente l'oggetto utente, controllando che il risultato sia corretto
- **testLoginFallito:** Il test verifica il ritorno null, il quale indica per questo test la presenza di credenziali errate
- **testLoginMatricolaNull:** Verifica che con il campo matricola vuota il controller ritorni null senza chiamare il DAO
- **testRegistrazioneMatricolaInvalida:** Il test verifica che il messaggio di ritorno contenga l'errore e che il metodo di registrazione non sia mai stato invocato sul DAO
- **testRegistrazioneSuccesso:** Il test verifica che il messaggio contenga il messaggio di operazione completata e che il DAO riceva tutti i parametri corretti

4.3 Test sul DAO

LoanDAOTest Verifica che il Data Access Object interagisca correttamente con il database

- **testCreaPrestito:** Verifica che il metodo di creazione del prestito generi un'istruzione di insert corretta verso la tabella `loans`, impostando i parametri di ID e ISBN correttamente
- **testChiudiPrestito:** Verifica la logica di chiusura del prestito. Il test assicura che venga generata una `update` che imposta la data di restituzione alla data corrente, filtrando correttamente per ISBN e ID, assicurandosi che il prestito fosse ancora aperto

LibraryItemDAOTest verifica il corretto funzionamento delle operazioni CRUD e di filtraggio sul catalogo bibliografico. Simula un flusso di integrazione reale per garantire che i dati vengano recuperati correttamente con tutte le loro associazioni

- **testInserimentoERecupero:** Verifica il ciclo di vita completo di una risorsa. Il test inserisce un nuovo libro nel database specificando tutti i dettagli e successivamente lo recupera tramite ISBN. L'asserzione finale controlla che l'oggetto restituito non sia nullo e che i dati recuperati corrispondano esattamente a quelli inseriti, confermando la corretta mappatura del database all'oggetto

- **testRicercaPerCorsoEAnno:** Verifica la logica di filtraggio necessaria per le ricerche degli studenti

CourseDAOTest Testa il l'interazione con la tabella `course_books`

- **testOttieniCorsi:** Configura il `ResultSet` (tabella di dati ottenuta come risultato di una query) per restituire i corsi, poi verifica che la lista ritornata contenga esattamente i corsi inseriti nell'ordine atteso.
- **testAggiungiAssociazioneEsistente:** verifica che il metodo di aggiunta di un'associazione già presente ritorni `false` senza eseguire l'operazione di `insert`
- **testGetCorsiByIsbn_LibroMultiCorso:** Simula un libro associato corsi diversi e verifica che il DAO restituisca tutte le associazioni ai corsi con la formattazione corretta, garantendo quindi la gestione della relazione multi-a-molti

4.4 Test sull'Observer

ObserverTest Testa il ciclo completo del pattern Observer implementato tramite `WaitingListManager`

- **testAttachNotifica:** Si ha uno studente come observer, lo registra nella coda di attesa per un ISBN specifico tramite il metodo di `attach`, poi invoca il metodo di prestito restituito. Il test verifica che il metodo sia stato invocato esattamente una volta, ricevendo l'ISBN corretto e un messaggio contenente il titolo restituito
- **testDetachNotificaMancante:** Registra due observer per uno stesso ISBN, poi rimuove il primo tramite metodo di `detach`. Dopo la chiamata al metodo di restituzione, verifica che solo il secondo observer riceva la notifica, confermando che il meccanismo di funziona correttamente e che un observer rimosso non riceve più messaggi.
- **testNotificheIsbnDiversi:** Registra due observer su due ISBN diversi, poi simula la restituzione solo per uno di essi, si Verifica che solo l'observer registrato su quello restituito riceva la notifica, mentre l'altro observer non viene mai invocato
- **testHaStudentiAttesa:** Verifica i metodi di ottenimento della lista di utenti in attesa e il numero di essi. Controlla che dopo la restituzione del libro la coda di attesa si svuoti